SOFTWARE ENGINEERING
& PROJECT MANAGEMENT

DAY 4

The Incremental Model

A student-facing chapter with practical examples, increment planning, benefits, limitations, and exercises

Module I: Introduction to Software Engineering
Topic: Software Process Models — Incremental Model
Duration: 1 hour

Running example: College Event Management System (CEMS)

This chapter explains how a software product can be delivered in smaller, useful releases, allowing users to gain value early and provide feedback before the entire system is complete.

1. Learning Objectives

By the end of this chapter, you should be able to:

- Define the Incremental Model and explain its purpose.
- Distinguish an increment from a complete project or a development phase.
- Describe how requirements, design, coding, and testing can be repeated for each increment.
- Plan a sequence of releases for a realistic system.
- Explain the benefits, limitations, and suitable use cases of incremental development.
- Compare incremental development with the Waterfall Model.

> **Key idea**
> In the Incremental Model, the product is built and delivered in a series of usable portions. Each increment adds agreed functionality to what has already been delivered.

2. Why Deliver Software in Increments?

Imagine a college needs an event management application before its annual festival. A complete system might include event listings, registration, notifications, QR-code attendance, feedback, certificates, and reports. If the team waits until every feature is finished, students may have no usable system until the end.

An incremental approach can release a basic but useful version first. The team can then add registration, notifications, attendance, and reporting in later increments. Users start receiving value earlier, and feedback can influence future increments.

This does not mean that the team skips planning, design, or testing. Each increment still needs appropriate requirements, design, implementation, and testing. The scope and level of detail may be planned for the product as a whole and refined for each increment.

3. What Is the Incremental Model?

The **Incremental Model** is a software development approach in which the system is developed, tested, and delivered in multiple increments. Each increment provides a defined set of features and extends the capabilities of the existing product.

A first increment may provide core functionality. Later increments add more capabilities until the planned product scope is delivered. Depending on the project, increments may be released to real users or demonstrated internally before public release.

> **Important distinction**
> An increment is a usable addition to the product, not simply a phase such as “coding” or “testing.” Each increment generally passes through relevant development and quality checks.

Overall product vision and priorities
↓
Increment 1: Plan → Design → Build → Test → Deliver
↓
Increment 2: Plan → Design → Build → Test → Deliver
↓

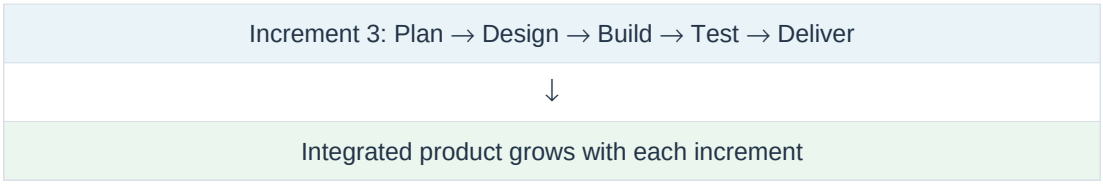

Figure 1. The product grows through successive usable increments. The exact number and size of increments depend on project scope and priorities.

4. How Does the Incremental Model Work?

Step 1 — Establish the Product Vision and Priorities

The team identifies the main problem, stakeholders, broad scope, constraints, and desired outcomes. Features are prioritized according to business value, risk, dependencies, and urgency.

For CEMS, event listings and registration may be more urgent than certificates or advanced analytics because students need to discover events and reserve places first.

Step 2 — Divide the Scope into Increments

The team groups related features into planned releases. Each increment should have a clear purpose and an outcome that can be reviewed or used. The team must also consider dependencies: registration depends on knowing which events exist, and attendance depends on registrations or participant lists.

Step 3 — Develop Each Increment

For the selected features, the team clarifies requirements, designs the solution, implements code, and tests it. The team checks that the new work functions correctly and does not break existing features.

Step 4 — Deliver and Collect Feedback

The increment is demonstrated or released to its intended audience. Users try the features, report problems, and share what is missing or confusing. Feedback helps the team adjust priorities for later increments.

Step 5 — Build on the Existing Product

The next increment extends the product. The team integrates new features with existing ones, performs integration and regression testing, updates documentation, and prepares the next release.

5. Worked Example: Increment Planning for CEMS

The college wants a system that helps students discover events and register online. Instead of waiting for all features to be complete, the project team plans a sequence of releases.

Increment	Features delivered	Value to users	Checks before delivery
1 — Core event catalogue	Event list, event details, date, venue, organizer contact.	Students can find accurate event information.	Correct event details, access, mobile layout, and basic usability.
2 — Registration	Login, register/cancel, capacity limits, confirmation.	Students can reserve places.	Duplicate registration, full events, invalid inputs, permissions.
3 — Notifications	Confirmation and event update notifications.	Students receive timely updates.	Correct recipient, message content, delivery failures.
4 — Attendance	Participant list, QR-code check-in, attendance status.	Organizers can record attendance efficiently.	Invalid or repeated scans, unauthorized access, record accuracy.
5 — Reports and certificates	Attendance reports, feedback summary, certificate generation.	Organizers can evaluate events and recognize participants.	Report accuracy, certificate eligibility, privacy, export format.

Planning note
The table is an example, not a mandatory release plan. A real team may combine or split increments based on deadlines, dependencies, capacity, architecture, and user priorities.

6. What Happens During One Increment?

A simple sequence for an increment is:

1. **Select scope:** agree which features and acceptance criteria belong in this increment.
2. **Clarify requirements:** confirm user needs, constraints, and edge cases.
3. **Design:** define data, interfaces, interactions, and integration with existing features.
4. **Implement:** develop the selected functionality and review the code.
5. **Test:** perform unit, integration, system, and regression checks as appropriate.
6. **Review and deliver:** demonstrate or release the working increment, document known issues, and collect feedback.

The process is repeated for the next increment. Teams do not necessarily repeat every activity with the same effort each time; the approach depends on the size and risk of the change.

7. Advantages of the Incremental Model

- **Early value:** users may begin using core features before the full system is complete.
- **Earlier feedback:** users can review working software and help refine later features.
- **Prioritization:** the team can deliver high-value features first.
- **Manageable scope:** a large project can be divided into smaller releases.
- **Risk visibility:** integration or usability issues may surface earlier than in a single end-of-project release.

- **Adaptability:** priorities can be reconsidered between increments, subject to architecture and change-control needs.

8. Limitations and Challenges

- **Architecture matters:** the team needs a design that can support future increments without repeated major restructuring.
- **Integration complexity:** new features must work with existing functionality and data.
- **Planning dependencies:** features cannot always be delivered in any order.
- **Regression risk:** changes can break previously working features, so repeat testing is important.
- **Scope creep:** continuous requests can expand the project unless scope and priorities are managed.
- **Release overhead:** multiple deployments may require extra coordination, documentation, and support.

9. When Is the Incremental Model Suitable?

It is often useful when the system can be divided into meaningful feature groups, early delivery has value, and users can review releases. Examples include a student portal, e-commerce site, event management system, or internal business application where features can be introduced in stages.

It may be harder to apply when the system cannot provide useful value until tightly coupled components are complete, when dependencies prevent safe separation, or when the architecture is not ready to support growth.

10. Incremental Model vs Waterfall Model

Aspect	Waterfall Model	Incremental Model
Delivery	Often delivers the integrated product after major phases are completed.	Delivers the product in planned usable portions.
Work pattern	Primarily sequential phases across the project.	Requirements/design/build/test activities recur for each increment.
User feedback	May arrive relatively late.	Can be gathered after each delivered or demonstrated increment.
Change handling	Late changes can require revisiting earlier outputs.	Priorities can be adjusted between increments, with impact analysis.
Main challenge	Early misunderstanding may be discovered late.	Managing architecture, integration, dependencies, and regression.

Do not confuse the two
Waterfall is primarily organized as a sequence of project phases. Incremental development organizes delivery into portions of product functionality; each portion still needs suitable analysis, design, implementation, and testing.

11. Classroom Activity: Plan Three Increments

Scenario: A department wants a Student Attendance Portal. It needs student records, attendance marking, attendance reports, and alerts for low attendance.

In small groups, decide:

1. Which features should be included in Increment 1, and why?
2. Which features depend on earlier features?
3. What should be tested before each increment is delivered?
4. What feedback would you ask students and faculty to provide?
5. Name one risk of delivering the features in your chosen order.

Expected reasoning: begin with a coherent core such as student/class records and attendance marking; reports and alerts depend on accurate attendance data. The best plan should explain priorities and dependencies rather than simply list features.

12. Knowledge Check

1. Define the Incremental Model.
2. What is an increment?
3. Why might an organization choose incremental development instead of waiting for one complete release?
4. Name two activities that should be repeated or performed for each increment.
5. Give two advantages and two limitations of the model.
6. Why are regression tests important when a new increment is added?
7. Can every feature be delivered in any order? Why or why not?
8. How does incremental delivery differ from Waterfall delivery?

Answer key

1. A development approach that delivers a system through successive usable portions. 2. A planned addition of functionality to the product. 3. To provide earlier value and gather feedback. 4. Examples: clarify requirements, design, implement, test, integrate, review. 5. Advantages: early value and feedback; limitations: integration complexity and regression risk. 6. To check that new changes have not broken existing features. 7. No; dependencies and architecture influence the order. 8. Incremental development delivers planned portions; Waterfall is primarily organized around sequential phases.

13. Key Terms

Term	Meaning
Increment	A planned portion of product functionality that extends the existing system.
Product vision	A high-level description of the product's purpose and intended value.
Feature priority	The relative importance assigned to a feature for planning and delivery.
Dependency	A relationship in which one feature or activity relies on another.
Integration testing	Testing whether connected modules or components work together.
Regression testing	Retesting existing functionality to check that changes have not caused unintended failures.
Release	A version of software made available to users or another intended audience.
Scope creep	Uncontrolled growth in project requirements or features.

14. Homework / Preparation for Day 5

For an online food ordering application, propose four increments. For each increment, write:

- The features included and the user value delivered.
- One dependency on an earlier increment, if applicable.
- Two test checks that should be completed before delivery.
- One question you would ask users after the release.

Next chapter
Day 5 will cover the Spiral Model, focusing on risk-driven development, risk analysis, prototyping, and repeated planning cycles.

Syllabus alignment: Module I — Introduction to Software Engineering; Software Process Models: Incremental. This chapter uses CEMS and a Student Attendance Portal as teaching examples.